orbits

I am building an interactive educational web platform that blends the clear, parameter-driven Kepler visualisation of the iwant2study Kepler model, the pedagogical animations from the UNL Kepler pages, and the hands-on multi-body sandbox feel of the PhET “Gravity and Orbits” simulation. The site will present a canonical Kepler two-body solver (ellipses, hyperbolae, conserved energy/Runge–Lenz vector, perihelion precession where applicable) with live controls for orbital elements and initial conditions, and layered on top of that will be a suite of perturbation experiments: add one or more additional masses, apply small changes to angular momentum, introduce weak non-Keplerian potentials, and explore nearly-integrable Hamiltonian regimes where quasi-periodic motion breaks down into chaotic scattering. A complementary module will implement a Rutherford-scattering analogue for comets — single-pass hyperbolic encounters with a central body plus a scattering perturbor — to demonstrate scattering cross sections, deflection angles, and sensitivity to impact parameter and incoming velocity. All simulations will be interactive, numerically robust, and designed for students to toggle between analytic conserved-quantity diagnostics and raw trajectory visualisations.

Target users are senior-freshman and above in theoretical physics and physical sciences at Trinity College Dublin (undergraduates and early postgraduates). I am a theoretical physics student building this tool to bridge the high-level analytic presentation in Landau & Lifshitz (Kepler problem, precession, perturbation theory) with hands-on numerical experiments that reveal how small parameter changes produce large dynamical consequences. A central research/teaching emphasis will be perturbation theory and chaos: in particular, studying regimes where the angular momentum of an individual body is not conserved (because of torques, multi-body interactions or non-central perturbations) even while total angular momentum of the isolated many-body system is respected; this contrast—between conserved integrals in the ideal Kepler problem and their breakdown under perturbations—is an explicit learning objective.

This will be a website that other students can access when professors share it with them on Blackboard as a link. And all the simulations will be in 2D.

Current Aim:

I am building an interactive educational web platform that blends the clear, parameter-driven Kepler visualisation of the iwant2study Kepler model, the pedagogical animations from the UNL Kepler pages, and the hands-on multi-body sandbox feel of the PhET “Gravity and Orbits” simulation. The site will present a canonical Kepler two-body solver (ellipses, hyperbolae, conserved energy/Runge–Lenz vector, perihelion precession where applicable) with live controls for orbital elements and initial conditions, and layered on top of that will be a suite of perturbation experiments: add one or more additional masses, apply small changes to angular momentum, introduce weak non-Keplerian potentials, and explore nearly-integrable Hamiltonian regimes where quasi-periodic motion breaks down into chaotic scattering. A complementary module will implement a Rutherford-scattering analogue for comets — single-pass hyperbolic encounters with a central body plus a scattering perturbor — to demonstrate scattering cross sections, deflection angles, and sensitivity to impact parameter and incoming velocity. All simulations will be interactive, numerically robust, and designed for students to toggle between analytic conserved-quantity diagnostics and raw trajectory visualisations.

Target users are senior-freshman and above in theoretical physics and physical sciences at Trinity College Dublin (undergraduates and early postgraduates). I am a theoretical physics student building this tool to bridge the high-level analytic presentation in Landau & Lifshitz (Kepler problem, precession, perturbation theory) with hands-on numerical experiments that reveal how small parameter changes produce large dynamical consequences. A central research/teaching emphasis will be perturbation theory and chaos: in particular, studying regimes where the angular momentum of an individual body is not conserved (because of torques, multi-body interactions or non-central perturbations) even while total angular momentum of the isolated many-body system is respected; this contrast—between conserved integrals in the ideal Kepler problem and their breakdown under perturbations—is an explicit learning objective.

This will be a website that other students can access when professors share it with them on Blackboard as a link. And all the simulations will be in 2D.

Later I’m going to add a third body orbiting the central mass (star) and perturbing the motion of the original planet, causing precession.

Environment

- Editor: **Visual Studio Code**
- Language: **TypeScript**
- Runtime: **Browser**
- Deployment: **GitHub Pages**
- Rendering: **HTML5 Canvas (2D)**
- Build tool: **Vite**

In windows powershell:
cd C:\Users\KnoxA\Projects\OrbitSimulations\orbit-sims
npm run dev

I now have a local TypeScript + Vite project set up for my orbit simulations, with Node and npm installed, and a dev server I can start anytime to view `http://localhost:5173` in the browser and see my site live-update as I edit code in VS Code.

Work Split:
- index.html (about 20–40 lines): defines only structure and UI. Contains the canvas element, sliders and buttons for masses, initial position and velocity, timestep or speed, and reset or pause. Includes the script tag loading src/main.ts. No physics, no rendering logic, no state updates.
- physics.ts (about 100–200 lines): contains all physics and numerics. Defines 2D vector types, constants, state variables (positions, velocities, masses), Newtonian gravity force law, equations of motion, and a numerical integrator (Velocity Verlet or RK4). Exports a function that advances the system by one timestep. No DOM access and no drawing code.
- main.ts (about 150–300 lines): orchestrates the app. Imports physics.ts, initializes state, sets up the canvas and coordinate scaling, runs the animation loop with requestAnimationFrame, draws bodies and trails, and connects UI controls to simulation parameters. No physics equations beyond calling physics.ts.
- style.css (about 30–60 lines): handles layout and appearance only. Styles the canvas, control panel, sliders, and buttons. No logic.

All computation runs client-side in the browser. No backend. Free hosting via GitHub Pages.

Coding Approach Choice:
Ranked methods (best → worst) for “Kepler + optional 3rd body / visible precession”

1. **Analytic Kepler base + Gauss / Lagrange planetary equations (perturbative element propagation)**
    - When to use: third body is a *small perturbation* (mass much smaller than central+primary or distant), you want clear precession/secular effects, and instant interactive recomputation when sliders change.
    - How it works: compute orbital elements of the perturbed pair (a,e,i,Ω,ω,M) from r,v; compute perturbing acceleration apert\mathbf{a}_\mathrm{pert}apert from the third body in the same inertial frame; evaluate Gauss planetary equations a˙,e˙,i˙,Ω˙,ω˙,M˙\dot a,\dot e,\dot i,\dot\Omega,\dot\omega,\dot Ma˙,e˙,i˙,Ω˙,ω˙,M˙ from apert\mathbf{a}_\mathrm{pert}apert decomposed into radial, transverse, normal components; integrate these ODEs with an inexpensive fixed-step integrator (or analytic averaging for secular trends).
    - Pros: very fast, preserves exact Kepler baseline between perturbations, shows clean precession terms and secular drift, great UX (sliders affect elements directly).
    - Cons: breaks down when perturbation is not small (close encounters, comparable mass). Requires careful choice of timestep and decomposition of perturbation components.
    - Implementation tips: keep the universal Kepler solver as drift step; update elements with Gauss eqns each physics step; use fixed physics dt + accumulator; provide option to toggle secular averaging (remove short-period terms) for long-term precession view.
2. **Wisdom–Holman / operator-splitting integrator (Kepler map + kicks)** — *best numeric compromise for hierarchical 3-body*
    - When to use: central mass dominates but the third body causes persistent perturbations (planetary regime), and you need long-term fidelity + speed.
    - How it works: split Hamiltonian H=HKepler+HpertH = H_\mathrm{Kepler} + H_\mathrm{pert}H=HKepler+Hpert. Integrate exactly the Kepler part for each body about the central mass (use analytic Kepler solver) and apply interaction “kicks” from the third body velocity updates. Typical scheme: Kick (dt/2) → Kepler drift (dt) via analytic solver → Kick (dt/2).
    - Pros: symplectic (good long-term behavior), high speed because analytic Kepler drift is cheap, directly supports multiple orbiting bodies. Excellent for moderate perturbations and museum-quality visualizations.
    - Cons: requires careful bookkeeping (which bodies are integrated around which primary), performance complexity if system becomes non-hierarchical. Not ideal for close encounters unless hybridized.
    - Implementation tips: implement Kepler drift using your universal variable solver for each primary–satellite pair; compute perturbation accelerations only from interactions not included in Kepler drift; use fixed dt and symplectic step.
3. **Analytic base + Gauss variational equations + numerical averaging / secular theory (Laplace–Lagrange)**
    - When to use: to extract secular frequencies, eigenmodes, and precession rates for long timescales; great for educational displays of forced/free precession.
    - How it works: linearize for small eccentricities/inclinations and compute secular matrix → eigenfrequencies. Or numerically average perturbing forces over fast angles to get slow evolution.
    - Pros: gives closed-form precession rates and teaching material (eigenmodes), cheap to compute.
    - Cons: limited parameter range (small e/i), not accurate for strong perturbations.
4. **Full symplectic N-body (high-order symplectic integrators)**
    - When to use: you want accurate N-body evolution without relying on Kepler hierarchy, or you want to support many bodies.
    - How it works: use fixed-step symplectic integrator (splitting into kinetic/potential) or higher-order symplectic Runge–Kutta methods for improved accuracy.
    - Pros: preserves phase-space structure, good for long horizons.
    - Cons: timestep constrained by shortest dynamical timescale; less efficient than WH for near-Kepler problems.
5. **Hybrid integrator: symplectic outside encounters + high-order adaptive (Bulirsch-Stoer/RK-F) during encounters**
    - When to use: interactions are usually weak but occasional close encounters occur.
    - How it works: detect when pairwise separation < threshold → switch to adaptive integrator for affected particles, then switch back.
    - Pros: combines long-term symplectic stability with accurate handling of close interactions.
    - Cons: complex implementation (state conversions, reversibility issues), tricky to keep global conservation properties.
6. **Regularized integrators (KS transform, Kustaanheimo–Stiefel, Chain regularization)**
    - When to use: you expect collisions or extremely close periapsis passages and want to avoid timestep blow-up.
    - Pros: handles singularities cleanly.
    - Cons: math-heavy and more effort; usually overkill for simple demo sites.
7. **Direct non-symplectic adaptive integrators (RK45, Dormand–Prince, Bulirsch–Stoer)** — *use sparingly*
    - Use for short high-accuracy runs or debugging; avoid for long-term interactive playback because energy drift destroys physical intuition.
8. **Naive Euler / RK2 / simple Verlet on Cartesian coords** — *worst for scientific realism*
    - Pros: trivial to implement.
    - Cons: unacceptable secular errors, bad energy / precession fidelity.

Concrete algorithm sketches

Perturbative (Gauss) update per physics step dt

1. Compute r,v of perturbed body relative to its focus.
2. Compute perturbing acceleration $\mathbf{a}_{\text{pert}} = \mathbf{a}_{\text{3rd}}$ (subtract central attraction included in Kepler governing mu).
3. Decompose $\mathbf{a}_{\text{pert}}$ into $(R,\ T,\ N)$ (radial, transverse, normal).
4. Evaluate Gauss equations:

$$\dot{a} = \frac{2}{n\sqrt{1-e^2}}\left[e\sin f\ R + \frac{p}{r}\ T\right], \quad \dot{e} = \frac{\sqrt{1-e^2}}{na}\left[\sin f\ R + \left(\cos f + \frac{e+\cos f}{1+e\cos f}\right)T\right]$$

(and so on for i,Ω,ω,M). Integrate these with fixed-step Euler/Heun or RK2 for the slow variables; then convert back to r,v via classical element→state transform. (Use small dt.)

Wisdom–Holman step (one dt)

- Kick: compute interaction accelerations from third body for each planet and update velocities by $v \leftarrow v + a_{\text{int}}\,(dt/2)$.
- Drift: for each body, propagate analytically under two-body Kepler (central mass) for dt using universal solver.
- Kick: recompute interaction accelerations and apply another half kick.
  This is symplectic and efficient.

Practical choices for web (TypeScript) implementation

- Implement universal Kepler solver in a separate module and unit-test against known cases (circle, ellipse, parabola, hyperbola).
- Implement element↔state transforms. Keep everything double precision (JS Number).
- For perturbative mode, precompute unit vectors for R,T,N each step for efficient Gauss evaluation.
- For WH integrator, choose a reference central mass (e.g., COM or the dominant central mass) and document assumptions.

- Provide visual toggles: show orbital elements, show perturbation vectors, show long-term residue (energy drift) so users see differences between methods.

Which method to pick as default for your use-case

- **Default mode:** Analytic Kepler + Gauss perturbative updates. Rationale: it gives exact Kepler motion when the third body is negligible, shows precession clearly, is very fast for interactive sliders, and gracefully degrades as perturbation grows (you can warn and switch to WH or hybrid if perturbation > threshold).

- **Secondary mode:** Wisdom–Holman symplectic for users who crank perturbation mass or want long integrations.

- **Advanced mode:** Hybrid with adaptive solver + regularization for research-grade accuracy and close encounters.

5) Kepler solver notes

- Use double precision math (JS numbers are double; fine).
- Implement robust root finder: Newton-Raphson with fallback to bisection if iterations diverge or step too large.
- Provide special-case solvers for e near 0 or e near 1 (use series expansions or universal variable approach).

UI & UX design suggestions (for physicist audience)

- Offer two modes in UI: **Exact two-body** (analytic) and **Numerical N-body** (symplectic). Let user toggle.
- Show conserved quantities: energy, angular momentum, eccentricity vector (plot e vector), and COM drift.
- Provide sliders/inputs for masses, initial positions/velocities, G, time span, timestep, integrator choice.
- Display orbital elements and allow changing `a,e,i,Ω,ω,M0` directly (recompute `r,v` via classical element→state transform).
- Visualize periapsis, apoapsis, focus point, and true anomaly vs time plots.
- For hyperbolic tracks show asymptotes and v∞.

Part of a series on

Astrodynamics

Orbital mechanics

Orbital elements hide

Apsis · Argument of periapsis · Eccentricity · Inclination · Mean anomaly · Orbital nodes · Semi-major axis · True anomaly

Types of two-body orbits by eccentricity show

Equations show

Celestial mechanics

Gravitational influences show

N-body orbits show

So what's the "solver" pipeline?

Minimal deterministic pipeline for an elliptic Kepler solver:

1. Input: a, e, τ (or M_0 at epoch), and t. wikipedia +1
2. Compute $M = n(t - \tau)$ with $n = \sqrt{\mu/a^3}$ and reduce M modulo 2π. ntrs.nasa +1
3. Solve $M = E - e\sin E$ for E (Newton/Halley/Danby/etc.). wikipedia +1
4. Compute r, (x, y) in orbital plane from E. wikipedia
5. Compute ν from E using the half-angle or atan2 formulas above. duncaneddy.github +1
6. (Optional) rotate from orbital plane to inertial coordinates using i, Ω, ω.

That's "analytic" in the sense: no ODE integration; just one root-find + closed forms. ntrs.nasa +1

The universal formulation rewrites the two-body propagation problem in terms of:

- A single anomaly χ (universal anomaly)
- A single parameter $\alpha = 1/a$
  - elliptic: $\alpha > 0$
  - parabolic: $\alpha = 0$
  - hyperbolic: $\alpha < 0$

Time evolution is given by solving:

$$\sqrt{\mu}(t - t_0) = r_0\chi + (\mathbf{r}_0 \cdot \mathbf{v}_0)\chi^2 C(\alpha\chi^2) + (1 - \alpha r_0)\chi^3 S(\alpha\chi^2)$$

where $C(z)$ and $S(z)$ are **Stumpff functions**, defined by convergent series that are **regular at** $z = 0$.

Key point:

> The parabolic case is **not special-cased.**
> It emerges smoothly as $\alpha \to 0$.

Perturbations and elemental variance [edit]

Main article: Perturbation (astronomy)

Unperturbed, two-body, Newtonian orbits are always conic sections, so the Keplerian elements define an unchanging ellipse, parabola, or hyperbola. Real orbits have perturbations, so a given set of Keplerian elements accurately describes an orbit only at the epoch. Evolution of the orbital elements takes place due to the gravitational pull of bodies other than the primary, the non-sphericity of the primary, atmospheric drag, relativistic effects, radiation pressure, electromagnetic forces, and so on.

Keplerian elements can often be used to produce useful predictions at times near the epoch. Alternatively, real trajectories can be modeled as a sequence of Keplerian orbits that osculate ("kiss" or touch) the real trajectory. They can also be described by the so-called planetary equations, differential equations which come in different forms developed by Lagrange, Gauss, Delaunay, Poincaré, or Hill.

Overall the code is mostly correct, but there are several runtime, numeric and robustness issues to be aware of.

- **DOM assumptions: many getElementById(...) calls use non-null assertions or cast to HTMLInputElement and will throw if those elements are missing or renamed (mu, initPos, drawEarth, showTrail, timeScale, start/stop/step/reset, ecc, eccInput, status). Add null checks or fail-safe defaults.**
- **Input validation: parseVec and parseFloat usages do not validate NaNs. parseVec will return NaN elements if the user types invalid numbers. resetFromUI and the velocity computation divide by rmag without guarding rmag === 0, causing divide-by-zero if initPos is (0,0).**
- **Newton solver robustness: solveUniversalKepler does not guard against df === 0 or produce a diagnostic if the solver fails to converge (it silently continues with last chi). That can yield NaN/incorrect results. Consider checking df, limiting dchi, and throwing or falling back after maxIter.**
- **Stumpff functions / small-z behavior: stumpffC/S use branch-wise closed forms but for very small |z| you may get cancellation / precision loss; using series expansions for small |z| improves stability.**
- **Potential numerical corner-cases: initial guess for chi and the fallback may be suboptimal for some hyperbolic/parabolic cases; rmag check after computing rvec is good but other intermediate divisions (e.g., r0 in vr0 = vdot(...)/r0) assume r0>0.**
- **Drawing/transform details: ctx.setTransform and font sizing are coupled with devicePixelRatio; the code works but font sizing and coordinates are sensitive to the transform. Also draw() is called both inside stepSimulation and again each animation frame, causing redundant draws.**
- **Minor: naming nitpick — parameter vr0vec is actually the velocity vector (v0vec) which is confusing.**

Fixes to consider: validate DOM inputs, guard against zero radii, handle df === 0 and non-convergence in Newton, clamp/validate parsed numbers, add small-z Taylor series in stumpff functions, and remove duplicate draw() calls for efficiency.

For the effective potential against radius graph, i want the user to have an energy slider along the U effective axis, changing the eccentricity of the orbit on the primary canvas in real time. i also want it so that if the user changes G M m or r0 in the left UI panel, this will change the height of the constant Energy line on the U effective graph. Also i want there to be a red circle on the U effective against r function symbolizing where the spacecraft is in its orbit, and moving corresponding to that in real time.

Saved Tabs and Links:

Startup:
file:///C:/Users/KnoxA/Downloads/locked_in_2.pdf
file:///C:/Users/KnoxA/Downloads/locked_in.pdf
file:///C:/Users/KnoxA/Downloads/internships.pdf
https://www.desmos.com/calculator/8j7npa9n5i
https://www.desmos.com/calculator/gxbh3bf2oy
https://www.perplexity.ai/search/i-now-have-a-local-typescript-di3c0mJ5SWSOzGEm0.nGWA
Add:
https://chatgpt.com/c/695d99b6-868c-832c-a1e2-fbc498856f3e
Background:
https://www.perplexity.ai/search/generate-10-separate-images-wh-iCdmwUEmSDC8q8mqoV8kDg?preview=1
https://user-gen-media-assets.s3.amazonaws.com/gemini_images/c1510f0e-e3c6-4aa0-b5cf-065825882441.png
https://user-gen-media-assets.s3.amazonaws.com/gemini_images/2f42a8a4-d0ed-40b1-a2a6-3ae1d0a37823.png
https://user-gen-media-assets.s3.amazonaws.com/gemini_images/0f62f437-9f8f-49a3-9416-e744c1d5448c.png
https://user-gen-media-assets.s3.amazonaws.com/gemini_images/e320cabf-7462-4306-8565-91d90afefc87.png
Simulation Sites:
https://astro.unl.edu/naap/pos/animations/kepler.html
https://phet.colorado.edu/sims/html/gravity-and-orbits/latest/gravity-and-orbits_all.html
https://www.geogebra.org/m/FW8SkHWh
https://javalab.org/en/gravity_en/
https://evgenii.com/blog/two-body-problem-simulator/
https://orbit-architect.ai-solutions.com/?_gl=1*imn7t1*_gcl_au*MzA0NzU2MzA3LjE3NTUwMTE2NjUuMzc5ODcxMjYzLjE3NTk4NTA5NDguMTc1OTg1MDk0Nw..
https://orbitalmechanics.info/
https://ssd.jpl.nasa.gov/tools/orbit_viewer.html
https://trisolarchaos.com/?n=3&m=1.7,9.8,2.8&p=-1.2290,0.1014,1.8170,2.0326,2.7300,-1.5710,-2.3973,2.3106,-2.1798&v=0.1450,-0.0052,0.0882,-0.2066,-0.0612,-0.0110,0.6351,0.2174,-0.0151&s=1.0&so=0.10&im=verlet&dt=5.00e-4&rt=1.0e-6&at=1.0e-8&bs=0.50&sf=0&sv=1&cm=free&kt=1&st=1&ag=0&tl=1500&cp=2.0314,4.7911,11.8554&ct=0.0000,0.0000,0.1234
https://javalab.org/en/three_body_problem_en/
Full Orbit:
https://www.perplexity.ai/search/import-style-css-let-canvas-ht-baPbmMXHQ2.f3bC5O4h_1g
Guides:
file:///C:/Users/KnoxA/Downloads/Orbit%20Simulator%20%E2%80%94%20Ui%20Template%20(index.html,%20Style.css,%20Main.pdf
https://www.stjarnhimlen.se/comp/ppcomp.html#4
https://proceedings.scipy.org/articles/majora-212e5952-015#orbit-propagation
https://css-tricks.com/creating-your-own-gravity-and-space-simulator/
https://www.perplexity.ai/search/show-me-where-they-talk-about-8ma_w_pNSf6qoqemsJTwGg
https://orbital-mechanics.space/time-since-periapsis-and-keplers-equation/universal-variables-example.html

https://orbital-mechanics.space/time-since-periapsis-and-keplers-equation/universal-lagrange-coefficients-example.html

3 Body System Addition:

https://chatgpt.com/c/694ab79b-a3d4-8331-988f-5c98ea0f93b5

https://upload.wikimedia.org/wikipedia/commons/5/5a/5_4_800_36_downscaled.gif

https://homepages.dias.ie/ydri/Goldstein.pdf

https://www.noel-friedrich.de/terminal/gui/simulate/?simulation=1d-3-masses-2-springs

https://labs.minutelabs.io/Chaotic-Planets/#planetarySystem=W3sieCI6LTcuODY5Njc0MjU3NTc4MDAzLCJ5IjotMC4yODE3MjIzNjQ2NTAyNTg4NSwidngiOjAuMDAwNzc5MzgwNTgwNjA3NDA2LCJ2eSI6MC4wMzY0MDI5MDM0Mjg4MTQzMSwibWFzcyI6MTAwLCJjb2xvciI6IiNlYWY1MTYiLCJwYXRoIjp0cnVlfSx7InigiOjEzMy4xMzAzMjU3NDI0MjE5LCJ5Ijo0LjcxODI3NzYzNTM0OTc0MiwidngiOi0wLjAxMjgzODIyODEwMTE0OTgzOSwidnkiOi0wLjU5NTQwMDg5MjU5NjY5MDUsIm1hc3MiOjYuMTQ2NTQ2ODg3NTQ5NjcxLCJjb2xvciI6IiNlYjcwMzIiLCJwYXRoIjp0cnVlfSx7InigiOi0zMTMuMjQzNjM1NzI3NTY3MywieSI6LTguMjg4NzgyNDkxMjgyOTMyLCJ2eCI6MC4wMDk3MjcxNzgzMzI3MjE3NzIsInZ5IjowLjE5MzY5MTYwMzUzMDUyNTkyLCJtYXNzIjowLjEsImNvbG9yIjoiI2U0ZDQyYyIsInBhdGgiOnRydWV9XQ%3D%3D

https://shhawkins.github.io/three-body-simulator/

https://www.perplexity.ai/search/in-my-universal-kepler-simulat-gd84TrsdQouGu9TiFCZ_Hw

https://www.perplexity.ai/search/how-difficult-complex-would-it-LyXydFVvRreNU.XsVhuoYw

https://www.perplexity.ai/search/in-which-orbital-mechanics-sys-J.KQc6zLTYKr_volJjczWw

Effective Potential Graph:

https://chatgpt.com/c/696a73cd-c81c-8327-b5f9-4d393b7a1d40

https://www.perplexity.ai/search/this-is-my-current-site-code-h-f14dE9KdSEW85Ogn5RexLQ

https://www.perplexity.ai/search/alright-heres-the-deal-buster-uIEgQqAPQW2X6riXL9zpEQ

Other: Chatgpt Stuff Saved:

Canonical Kepler solver design
How should I structure a numerically stable 2D Kepler two-body solver that allows real-time switching between orbital-element parametrisation (a,e,ω)(a,e,\omega)(a,e,ω) and Cartesian initial conditions (r0,v0)(\mathbf{r}_0,\mathbf{v}_0)(r0,v0), while explicitly tracking conserved quantities (energy, angular momentum, Runge–Lenz vector) and their numerical drift?
Perturbations and near-integrability
What is a clean minimal set of perturbations (e.g. weak third body, 1/r+ϵr21/r + \epsilon r^21/r+ϵr2 potential, time-dependent torque) that best demonstrates the breakdown of Kepler integrability, and how can these be formulated Hamiltonian-first so students can see which symmetries are being broken?
Chaos diagnostics for students
Which chaos indicators are pedagogically effective and computationally cheap enough for a browser-based simulation (e.g. Poincaré sections, Lyapunov exponents, frequency-map analysis), and how should they be presented alongside raw trajectories to avoid black-box "chaos spotting"?
Angular momentum non-conservation at the subsystem level
How can I design multi-body or non-central-force scenarios where individual orbital angular momentum is not conserved but total system angular momentum is, and what visual or quantitative diagnostics best make this distinction explicit?
Rutherford / gravitational scattering module
How should I implement a 2D hyperbolic scattering experiment that connects analytic Rutherford-style deflection formulas θ(b,v∞)\theta(b,v_\infty)θ(b,v∞) with numerical trajectories, including how to define and visualise scattering cross sections and sensitivity to impact parameter?

1) Realistic planetary perturbations

- In reality, apsidal precession from another planet is very small.
- For example, Jupiter perturbs Earth's orbit, but the perihelion shift per year is tiny (~0.116″/yr).
- Typical planetary masses and distances produce barely noticeable precession over a few orbits.

---

2) Making precession visible in a simulation

To see precession clearly in a few orbits, you need to exaggerate the effect:

1. Increase the perturbing planet's mass (m2m_2m2)
    - It can be 10–100× the mass of the original planet (m1m_1m1) for noticeable effect.
    - Larger masses increase the precession rate proportionally.
2. Decrease the distance between the planets (a2a_2a2)
    - Place P2P_2P2 closer than realistic planetary spacing.
    - The gravitational influence scales roughly as F~m2/∣r1−r2∣2F \sim m_2 / |\mathbf{r}_1 - \mathbf{r}_2|^2F~m2/∣r1−r2∣2, so smaller separation dramatically increases precession.
3. Balance for stability
    - Avoid making m2m_2m2 or a2a_2a2 extreme enough to unbind P1P_1P1.
    - Aim for a range where P1P_1P1 remains in a bound, elliptical orbit but precesses visibly within a few simulated orbits.

---

3) Practical guideline for your simulation

- Mass ratio: m2~10–100×m1m_2 \sim 10\text{–}100 \times m_1m2~10–100×m1
- Distance: a2~1.2–3×a1a_2 \sim 1.2\text{–}3 \times a_1a2~1.2–3×a1 (slightly outside or inside the primary planet's orbit)

This range ensures strong perturbation while keeping orbits reasonably stable. You can let users adjust m2m_2m2 and a2a_2a2 dynamically for dramatic visual precession.